INVENTORY MANAGEMENT SYSTEM

Technical Architecture & Backend Specification

Version 1.0 • Spring Boot • Hexagonal Architecture • DDD

1. Project Overview

The Inventory Management System (IMS) is a backend service built with Spring Boot that enables merchants and operators to manage physical stock across multiple markets and storage locations. It provides full lifecycle management of items — from storage, to market allocation, through sales tracking, to final reconciliation when a market closes.

This specification covers domain modelling, API contracts, architecture decisions, infrastructure setup, and deployment configuration.

1.1 Business Capabilities

- Track item stock levels and physical positions in storage
- Create and manage markets with open/close lifecycle
- Shift items from storage to markets, reducing storage stock
- Increment and decrement stock at markets (sales and returns)
- Assign item prices per market
- Record and audit all stock transactions
- View sales and stock summaries per market
- Cross-market and cross-location reporting with filtering
- Close markets and reconcile stock back to storage

1.2 Technology Stack

Concern	Technology
Language & Runtime	Java 21 (LTS)
Framework	Spring Boot 3.3.x
Build System	Maven (multi-module)
Persistence	PostgreSQL 16 (primary), Flyway (migrations)
Messaging	Apache Kafka 3.7
Containerization	Docker + Docker Compose
API Documentation	SpringDoc OpenAPI 3 (Swagger UI)
Logging	SLF4J + Logback, structured JSON logs
Cross-cutting Concerns	Spring AOP
Architecture Style	Hexagonal (Ports & Adapters) + Onion + DDD
Service Initializer	Spring Initializr (start.spring.io)

2. Architecture

2.1 Hexagonal Architecture (Ports & Adapters)

The application is structured around a domain core that is completely isolated from infrastructure concerns. All interactions with the outside world pass through well-defined ports (interfaces), which are implemented by adapters.

Layer	Responsibility	Examples
Domain Core	Business rules, entities, value objects, domain events, domain services	Item, Market, Transaction, StockLevel
Application	Use cases, command/query handlers, port definitions	ShiftItemToMarketUseCase, CloseMarketUseCase
Inbound Adapters	Accept external requests and translate to application commands	REST Controllers, Kafka Consumers
Outbound Adapters	Implement repository and messaging ports	JPA Repositories, Kafka Producers
Infrastructure	Framework config, Kafka config, DB config, AOP, logging	KafkaConfig, FlywayConfig, LoggingAspect

2.2 Multi-Module Maven Structure

The project is split into discrete Maven modules to enforce layer boundaries at compile time. No module may import from a module that depends on it.

```
ims-parent/                          (Parent POM — dependency management)
  ims-domain/                        (Pure domain: entities, value objects, events,
ports)
  ims-application/                   (Use cases, application services — depends on
domain)
  ims-infrastructure/                (JPA, Kafka, Flyway, AOP — depends on application)
  ims-api/                           (REST controllers, DTOs, OpenAPI — depends on
application)
  ims-bootstrap/                     (Spring Boot main class, wires all modules together)
```

Note: *ims-domain must have zero Spring or JPA dependencies. It is plain Java only.*

2.3 Onion Structure within each Module

Within each module, packages follow the onion layers: domain model in the centre, application services surrounding it, and adapters at the edges. Dependencies always point inward.

3. Domain Model

3.1 Core Aggregates & Entities

3.1.1 Item

Represents a physical product that can be stored and sold.

Field	Type	Description
id	UUID	Primary identifier
sku	String	Stock keeping unit code (unique)
name	String	Product name
description	String	Optional description
category	String	Product category
defaultPrice	Money	Default price value object (amount + currency)
storagePosition	StoragePosition	Value object: shelf, row, column in storage
totalStorageStock	int	Current quantity in storage
createdAt	Instant	Creation timestamp
updatedAt	Instant	Last modification timestamp

3.1.2 Market

Represents a sales market or event where items are sold. A market has an open/close lifecycle.

Field	Type	Description
id	UUID	Primary identifier
name	String	Market name
place	String	Physical location / venue name
openDate	LocalDate	Date when the market opens
closeDate	LocalDate	Date when the market closes
status	MarketStatus	Enum: SCHEDULED, OPEN, CLOSED
createdAt	Instant	Creation timestamp

MarketStatus transitions:

- SCHEDULED → OPEN when the market is formally opened
- OPEN → CLOSED when the market is closed (triggers stock reconciliation)

3.1.3 MarketItem

Represents an item allocated to a specific market. This is a child entity of Market.

Field	Type	Description
id	UUID	Primary identifier
marketId	UUID	Parent market
itemId	UUID	Reference to the item
allocatedStock	int	Units shifted to this market
currentStock	int	Remaining stock at market (after sales)
marketPrice	Money	Price for this item at this market (overrides default)

3.1.4 Transaction

An immutable audit record of every stock movement. Transactions are append-only and are never deleted.

Field	Type	Description
id	UUID	Primary identifier
marketId	UUID	Market where the transaction occurred (nullable for storage ops)
itemId	UUID	Item involved
type	TransactionType	Enum: SHIFT_TO_MARKET, SALE, RETURN, STOCK_ADJUSTMENT, MARKET_CLOSE_RECONCILE
quantityDelta	int	Positive = stock added, Negative = stock removed
stockBefore	int	Stock level before the transaction
stockAfter	int	Stock level after the transaction
note	String	Optional operator note
occurredAt	Instant	When the transaction occurred
createdBy	String	User or system that triggered it

3.2 Value Objects

Value Object	Fields	Rules
Money	amount: BigDecimal, currency: String (ISO 4217)	amount >= 0; currency must be 3-letter code

Value Object	Fields	Rules
StoragePosition	zone: String, shelf: String, row: int, column: int	All fields required; zone is alphanumeric
StockLevel	quantity: int	quantity >= 0 always enforced by domain
DateRange	from: LocalDate, to: LocalDate	from must not be after to

3.3 Domain Events

Domain events are raised by aggregates and published via the outbound messaging port. All events extend a base DomainEvent class carrying eventId (UUID), occurredAt (Instant), and aggregateId.

Event	Trigger	Key Payload Fields
ItemCreatedEvent	New item registered	itemId, sku, name
ItemStockAdjustedEvent	Storage stock manually changed	itemId, delta, newStockLevel
MarketOpenedEvent	Market status set to OPEN	marketId, name, place, openDate
ItemShiftedToMarketEvent	Stock moved to market	marketId, itemId, quantity, remainingStorage
MarketStockIncrementedEvent	Stock incremented at market	marketId, itemId, delta, newStock
MarketStockDecrementedEvent	Stock decremented at market (sale)	marketId, itemId, delta, newStock
MarketClosedEvent	Market closed and reconciled	marketId, reconciledItems: list of {itemId, returnedQty}

4. Application Layer — Use Cases

4.1 Item Management

Use Case	Command / Query	Description
RegisterItemUseCase	RegisterItemCommand	Create a new item with SKU, name, category, default price, storage position, and initial stock
UpdateItemUseCase	UpdateItemCommand	Update item details (not stock)
AdjustStorageStockUseCase	AdjustStorageStockCommand	Increment or decrement storage stock; creates a STOCK_ADJUSTMENT transaction
GetItemQuery	GetItemQuery(id)	Fetch a single item with current storage stock
ListItemsQuery	ListItemsQuery(filters)	Paginated list of items with optional category/SKU filter

4.2 Market Management

Use Case	Command / Query	Description
CreateMarketUseCase	CreateMarketCommand	Create a market with name, place, open date, close date
OpenMarketUseCase	OpenMarketCommand(marketId)	Transition market to OPEN; publishes MarketOpenedEvent
CloseMarketUseCase	CloseMarketCommand(marketId)	Closes market; returns all remaining marketItem stock to storage; publishes MarketClosedEvent and creates MARKET_CLOSE_RECONCILE transactions
GetMarketQuery	GetMarketQuery(marketId)	Fetch market details and current status
ListMarketsQuery	ListMarketsQuery(filters)	Paginated markets; filter by status, place, date range

4.3 Market Stock Operations

Use Case	Command / Query	Description
ShiftItemToMarketUseCase	ShiftItemToMarketCommand	Move N units of an item from storage to a market; deducts storage stock; creates SHIFT_TO_MARKET transaction

Use Case	Command / Query	Description
IncrementMarketStockUseCase	IncrementMarketStockCommand	Add stock to a market item (e.g. restocking); creates a positive transaction
DecrementMarketStockUseCase	DecrementMarketStockCommand	Reduce stock at market (record a sale or write-off); creates a SALE transaction
SetMarketItemPriceUseCase	SetMarketItemPriceCommand	Set or override the price for a specific item at a specific market

4.4 Reporting Queries

Use Case	Output
GetMarketSummaryQuery(marketId)	Sales total, revenue, current stock levels per item, transactions list for one market
GetAllMarketsSummaryQuery(filters)	Aggregated summary across all markets; filterable by marketId, place, status, date range
GetTransactionHistoryQuery(marketId, itemId)	Paginated transaction log for a market/item combination
GetStorageSummaryQuery()	Current storage stock levels for all items with position

5. Ports (Interfaces)

5.1 Inbound Ports (Driven by Adapters)

Defined in ims-application. Implemented by application services.

- ItemCommandPort — exposes registerItem, updateItem, adjustStorageStock
- MarketCommandPort — exposes createMarket, openMarket, closeMarket
- MarketStockCommandPort — exposes shiftItem, incrementStock, decrementStock, setPrice
- ItemQueryPort — exposes getItem, listItems
- MarketQueryPort — exposes getMarket, listMarkets, getMarketSummary, getAllMarketsSummary
- TransactionQueryPort — exposes getTransactionHistory

5.2 Outbound Ports (Implemented by Adapters)

Defined in ims-application. Implemented in ims-infrastructure.

- ItemRepositoryPort — save, findById, findBySku, findAll
- MarketRepositoryPort — save, findById, findAll(filters)
- MarketItemRepositoryPort — save, findByMarketIdAndItemId, findAllByMarketId
- TransactionRepositoryPort — save, findByMarketId, findByItemId, findByMarketIdAndItemId
- DomainEventPublisherPort — publish(DomainEvent)
- StorageStockPort — addStock(itemId, qty), deductStock(itemId, qty), getStock(itemId)

6. REST API Specification

6.1 Base Configuration

Property	Value
Base path	/api/v1
Content-Type	application/json
Authentication	None (v1 — add Spring Security in future)
Pagination	?page=0&size=20&sort=createdAt,desc
Error format	RFC 7807 Problem Details (application/problem+json)

6.2 Item Endpoints

Method	Path	Description	Status
POST	/items	Register a new item	201 Created
GET	/items	List all items (paginated)	200 OK
GET	/items/{id}	Get item by ID	200 OK
PUT	/items/{id}	Update item metadata	200 OK
PATCH	/items/{id}/stock	Adjust storage stock (body: {delta: int, note: string})	200 OK
GET	/items/{id}/transactions	Item transaction history	200 OK

6.3 Market Endpoints

Method	Path	Description	Status
POST	/markets	Create a new market	201 Created
GET	/markets	List markets (filter: place, status, from, to)	200 OK
GET	/markets/{id}	Get market details	200 OK
POST	/markets/{id}/open	Open a scheduled market	200 OK
POST	/markets/{id}/close	Close market and reconcile stock	200 OK
GET	/markets/{id}/summary	Sales and stock summary for a market	200 OK
GET	/markets/summary	Aggregate summary (filter: marketId, place, status)	200 OK

6.4 Market Stock Endpoints

Method	Path	Description	Status
POST	/markets/{id}/items	Shift item to market {itemId, quantity}	201 Created
GET	/markets/{id}/items	List all items at market with stock	200 OK
PATCH	/markets/{id}/items/{itemId}/increment	Increment stock at market {quantity}	200 OK
PATCH	/markets/{id}/items/{itemId}/decrement	Decrement stock at market {quantity, note}	200 OK
PUT	/markets/{id}/items/{itemId}/price	Set item price at market {amount, currency}	200 OK
GET	/markets/{id}/items/{itemId}/transactions	Transactions for item at market	200 OK

7. Database Schema

7.1 Tables Overview

Table	Primary Key	Description
items	id (UUID)	Core item catalogue
storage_positions	item_id (FK)	One-to-one: storage location per item
markets	id (UUID)	Market definitions
market_items	id (UUID)	Item allocations per market (unique: market_id + item_id)
transactions	id (UUID)	Append-only transaction log
outbox_events	id (UUID)	Transactional outbox for domain event publishing

7.2 Key Constraints & Indexes

- items.sku: UNIQUE
- market_items: UNIQUE(market_id, item_id)
- transactions: INDEX(market_id), INDEX(item_id), INDEX(occurred_at)
- outbox_events: INDEX(status) for polling unpublished events
- All FKs enforce referential integrity with ON DELETE RESTRICT

7.3 Flyway Migrations

All schema changes are managed via Flyway versioned migrations in ims-infrastructure/src/main/resources/db/migration. Naming convention: V{version}__{description}.sql, e.g. V001__create_items_table.sql.

Note: *The outbox pattern is used to guarantee at-least-once delivery of domain events to Kafka without distributed transactions.*

8. Messaging with Apache Kafka

8.1 Topics

Topic	Partitions	Retention	Events Published
ims.items	3	7 days	ItemCreatedEvent, ItemStockAdjustedEvent
ims.markets	3	7 days	MarketOpenedEvent, MarketClosedEvent
ims.market-stock	6	7 days	ItemShiftedToMarketEvent, MarketStockIncrementedEvent, MarketStockDecrementedEvent

8.2 Outbox Pattern

To prevent loss of domain events during failures, the system uses the Transactional Outbox Pattern:

- When a use case commits a database transaction, domain events are persisted to the outbox_events table in the same transaction.
- A scheduled poller (OutboxPublisherJob) runs every 500ms, reads unpublished events, publishes them to Kafka, and marks them as published.
- This guarantees exactly-once delivery at the database boundary and at-least-once at the Kafka boundary.

8.3 Event Envelope

All events are serialised as JSON and wrapped in a standard envelope:

```
{
  "eventId": "<uuid>",
  "eventType": "ItemCreatedEvent",
  "aggregateId": "<uuid>",
  "occurredAt": "2025-06-01T10:30:00Z",
  "payload": { ... }
}
```

9. Cross-Cutting Concerns

9.1 Aspect-Oriented Programming (AOP)

Spring AOP is used to apply cross-cutting behaviour declaratively without polluting business logic.

Aspect	Pointcut	Behaviour
LoggingAspect	All public methods in application and domain packages	Logs method entry/exit with arguments and execution time at DEBUG level
ExceptionHandlingAspect	All use case methods	Catches domain exceptions, wraps in structured error response, logs at WARN or ERROR
TransactionAuditAspect	All methods annotated with @Transactional	Logs transaction start/commit/rollback with correlation ID
PerformanceAspect	Methods annotated with @Monitored	Logs a WARN if execution exceeds configurable threshold (default: 2000ms)

9.2 Logging Strategy

Property	Detail
Format	Structured JSON (Logback + logstash-logback-encoder)
Correlation ID	Generated per request (MDC key: correlationId), propagated through Kafka headers
Log Levels	ERROR: unhandled exceptions; WARN: business rule violations; INFO: request/response, major state changes; DEBUG: method traces
Sensitive Data	Prices and quantities logged; no PII by default
Output	Console (Docker log driver captures to centralized logging)

10. Containerization & Docker Compose

10.1 Dockerfile (ims-bootstrap)

Multi-stage build to minimize image size:

```
Stage 1 (builder): eclipse-temurin:21-jdk-alpine
  — runs mvn package -DskipTests
Stage 2 (runtime): eclipse-temurin:21-jre-alpine
  — copies the fat JAR from stage 1
  — EXPOSE 8080
  — ENTRYPOINT ["java", "-jar", "/app/ims-bootstrap.jar"]
```

10.2 Docker Compose Services

Service	Image	Ports	Purpose
ims-app	Built from ./Dockerfile	8080:8080	Spring Boot application
postgres	postgres:16-alpine	5432:5432	PostgreSQL database
zookeeper	confluentinc/cp-zookeeper:7.6	2181:2181	Kafka coordinator
kafka	confluentinc/cp-kafka:7.6	9092:9092	Message broker
kafka-ui	provectuslabs/kafka-ui:latest	8090:8080	Kafka management UI (dev only)

10.3 Environment Variables

Variable	Default	Description
DB_HOST	postgres	PostgreSQL hostname
DB_PORT	5432	PostgreSQL port
DB_NAME	ims	Database name
DB_USERNAME	ims_user	Database username
DB_PASSWORD	changeme	Database password
KAFKA_BOOTSTRAP	kafka:9092	Kafka bootstrap server
APP_PORT	8080	Application HTTP port
LOG_LEVEL	INFO	Root log level
OUTBOX_POLL_MS	500	Outbox poller interval (ms)

Note: *Secrets (passwords) must be supplied via environment variables or Docker secrets and must never be committed to source control.*

11. Spring Initializr Configuration

11.1 Spring Initializr Settings

Use https://start.spring.io to bootstrap the ims-bootstrap module with the following settings:

Setting	Value
Project	Maven Project
Language	Java
Spring Boot Version	3.3.x (latest stable)
Group	com.ims
Artifact	ims-bootstrap
Packaging	Jar
Java Version	21

11.2 Dependencies to Select

Dependency	Spring Initializr Label	Used For
spring-boot-starter-web	Spring Web	REST API
spring-boot-starter-data-jpa	Spring Data JPA	ORM and repositories
spring-boot-starter-aop	Spring AOP	Cross-cutting concerns
spring-boot-starter-actuator	Spring Boot Actuator	Health checks and metrics
spring-boot-starter-validation	Validation	Bean Validation (JSR-380)
spring-kafka	Spring for Apache Kafka	Kafka producer/consumer
flyway-core	Flyway Migration	Database migrations
postgresql	PostgreSQL Driver	JDBC driver
springdoc-openapi-starter-webmvc-ui	(Add manually in POM)	OpenAPI / Swagger UI
logstash-logback-encoder	(Add manually in POM)	Structured JSON logging
lombok	Lombok	Reduce boilerplate

Note: *ims-domain and ims-application are created as plain Maven modules (no Spring Boot plugin) and added as dependencies in the parent POM.*

12. API Documentation

API documentation is auto-generated by SpringDoc OpenAPI 3. The Swagger UI is accessible at:

```
http://localhost:8080/swagger-ui.html
http://localhost:8080/v3/api-docs  (raw OpenAPI JSON)
```

All controllers must be annotated with @Tag (group), @Operation (endpoint description), and @ApiResponse (response codes). DTOs must use @Schema annotations for field descriptions and examples.

12.1 OpenAPI Configuration

- Title: Inventory Management System API
- Version: v1
- Contact: ims-support@example.com
- License: Proprietary
- Servers: http://localhost:8080 (local), https://ims.example.com (production)

13. Business Rules Summary

Rule	Enforcement Layer
Storage stock may never go below zero	Domain — StockLevel value object
Market stock may never go below zero	Domain — MarketItem aggregate
Cannot shift items to a CLOSED market	Domain — Market aggregate
Cannot shift items to a SCHEDULED market	Domain — Market aggregate
Cannot decrement more units than currently available at market	Domain — MarketItem aggregate
Closing a market returns all remaining stock to storage	Application — CloseMarketUseCase
Transactions are append-only (immutable after creation)	Infrastructure — JPA entity with no setters on immutable fields
Market price for an item defaults to item default price if not overridden	Application — SetMarketItemPriceUseCase
A SKU must be globally unique across all items	Database — UNIQUE constraint + domain check
All domain events must be published via the outbox (no direct Kafka calls from domain)	Architecture — outbound port boundary

14. Future Considerations (Out of Scope v1)

- Authentication and authorisation (Spring Security + JWT / OAuth2)
- Multi-tenancy support
- Unit and integration test suites (to be specified separately)
- Prometheus metrics endpoint and Grafana dashboards
- Event sourcing for full audit trail
- Read model (CQRS projection) for reporting queries
- CI/CD pipeline (GitHub Actions / GitLab CI)